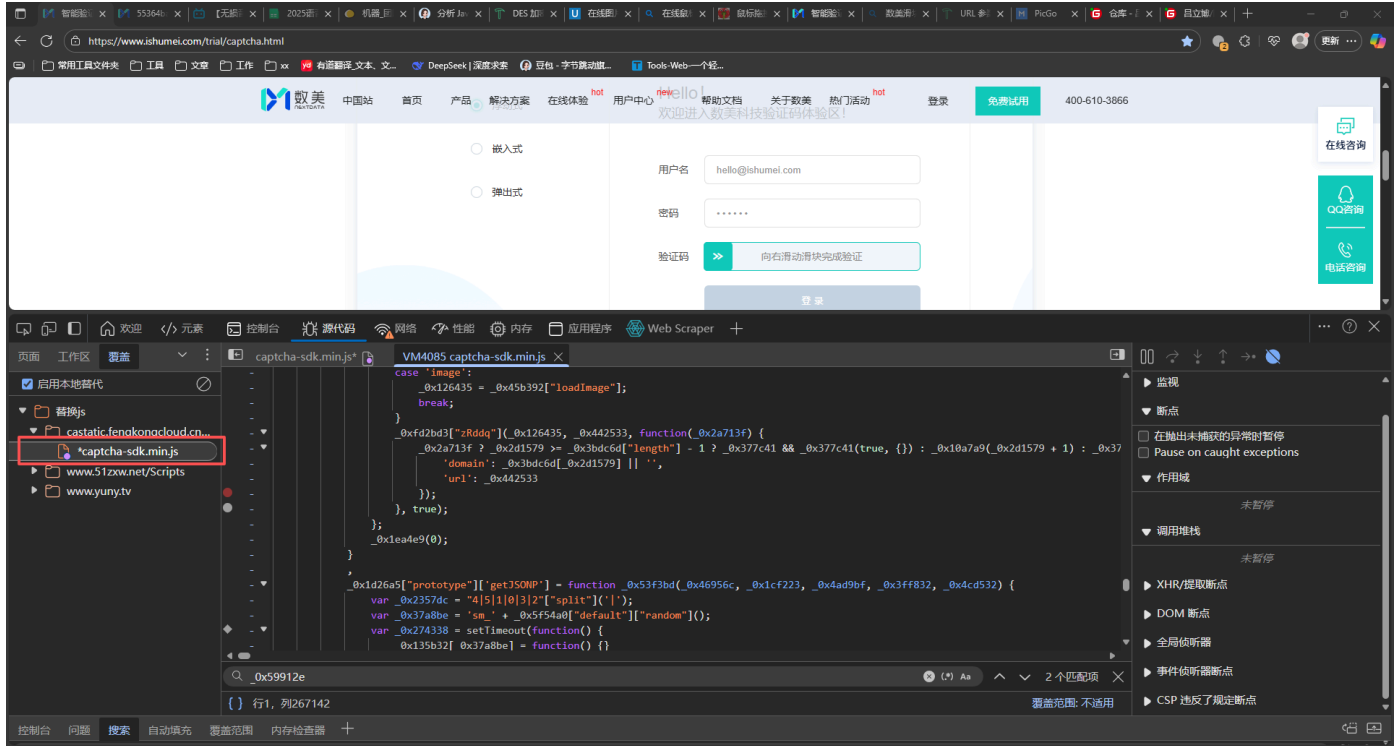
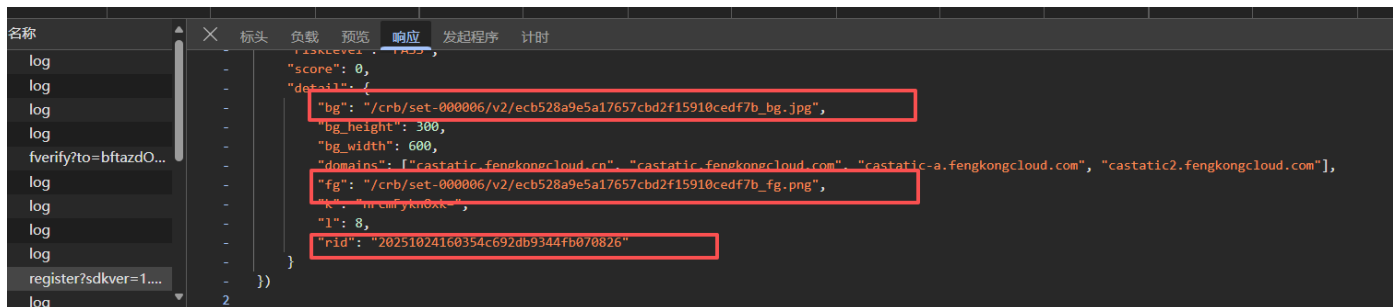
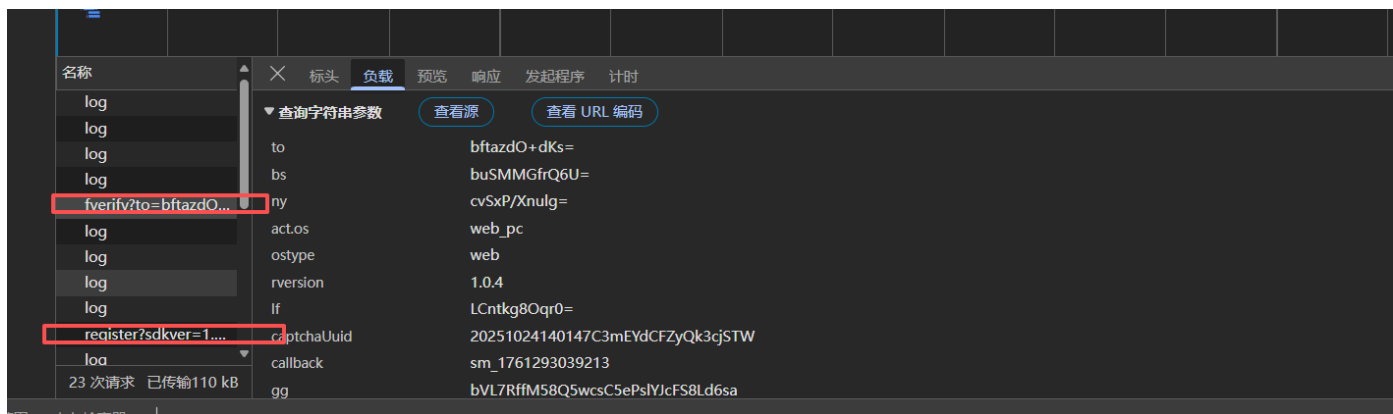


数美滑块

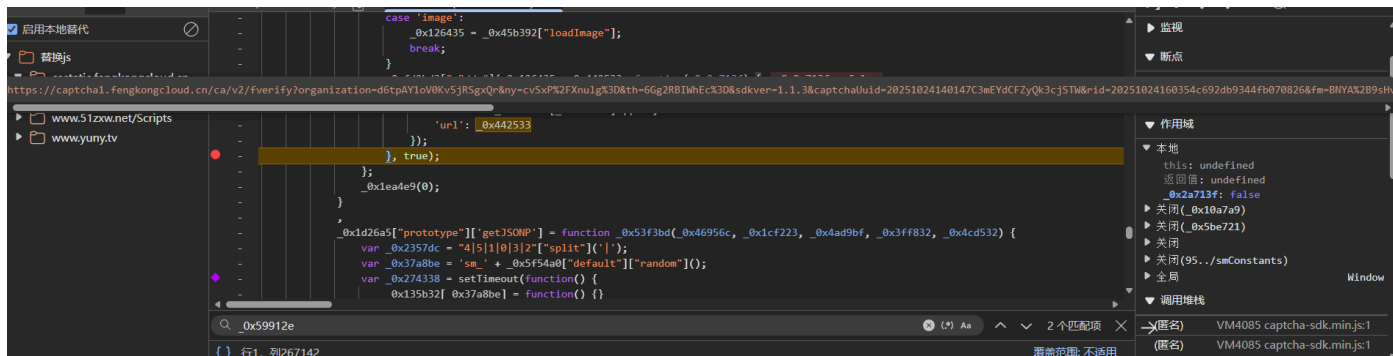
1.打开数美官网（[智能验证码体验_图片验证码_数美科技](#)）,找到浮动式滑块,替换js文件.



2.滑过一次滑块分析请求,可以看到,有两个请求是特殊的,观看请求可以看出,第一个是过滑块的,第二个是请求滑块图片的。



3.找到过滑块请求，查看调用堆栈，打断点，跟栈。可以看到url,拿去分析url的组成。这个就是过滑块请求的url。



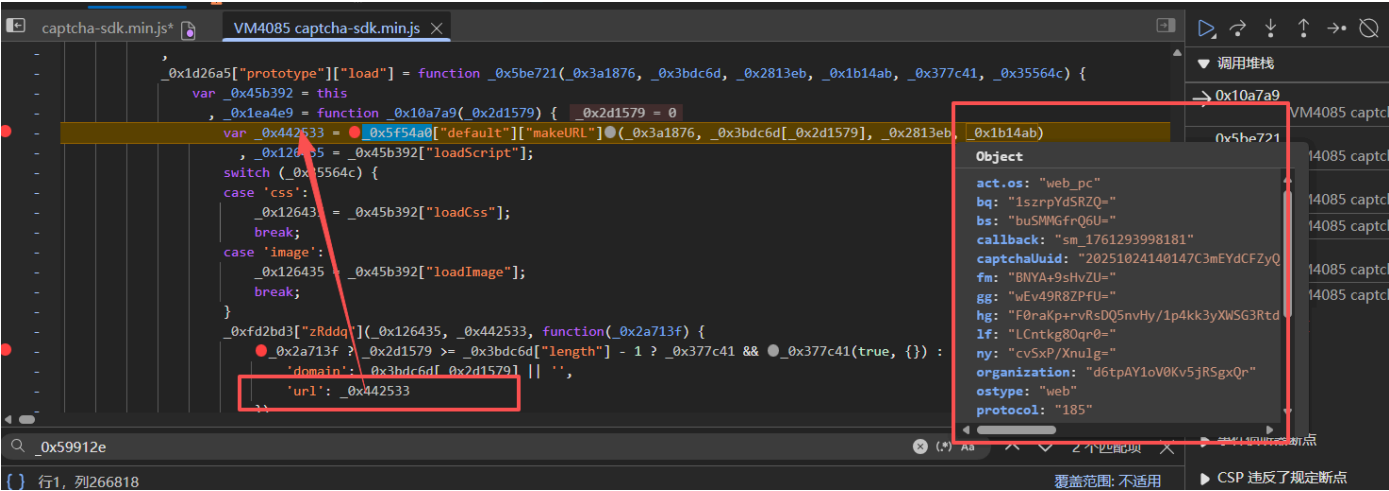
4.拿两次的过滑块的请求去对比，，我们搜一两个，可以看到是顺序不一致导致的差异，直接丢给ai，可以看到两次有5个差异。接下来暂时主要分析这5个参数。



以下是两段参数的差异对比及关键分析：

参数名	第一段值	第二段值	差异说明
rid	20251024160354c692db9344fb070826	20251024153305e17790f2be73d7509b	每次请求的随机 ID 不同（防重放）
callback	sm_1761293346498	sm_1761293039213	回调函数的时间戳不同
gg	MkHpZg9JP5TqjJPkm+ITYBswXg8Sf3y	bVL7Rffm58Q5wsc5SePs1YJcFS8Ld6sa	疑似轨迹或环境信息的加密值不同
th	6Gg2RB1wEc=	VamN/cFcYPU=	可能关联滑块背景或验证逻辑的参数不同
hg	(较短的加密串)	(极长的加密串，包含轨迹细节)	第二段 hg 是核心轨迹加密数据，长度和内容差异体现了鼠标轨迹的真实交互

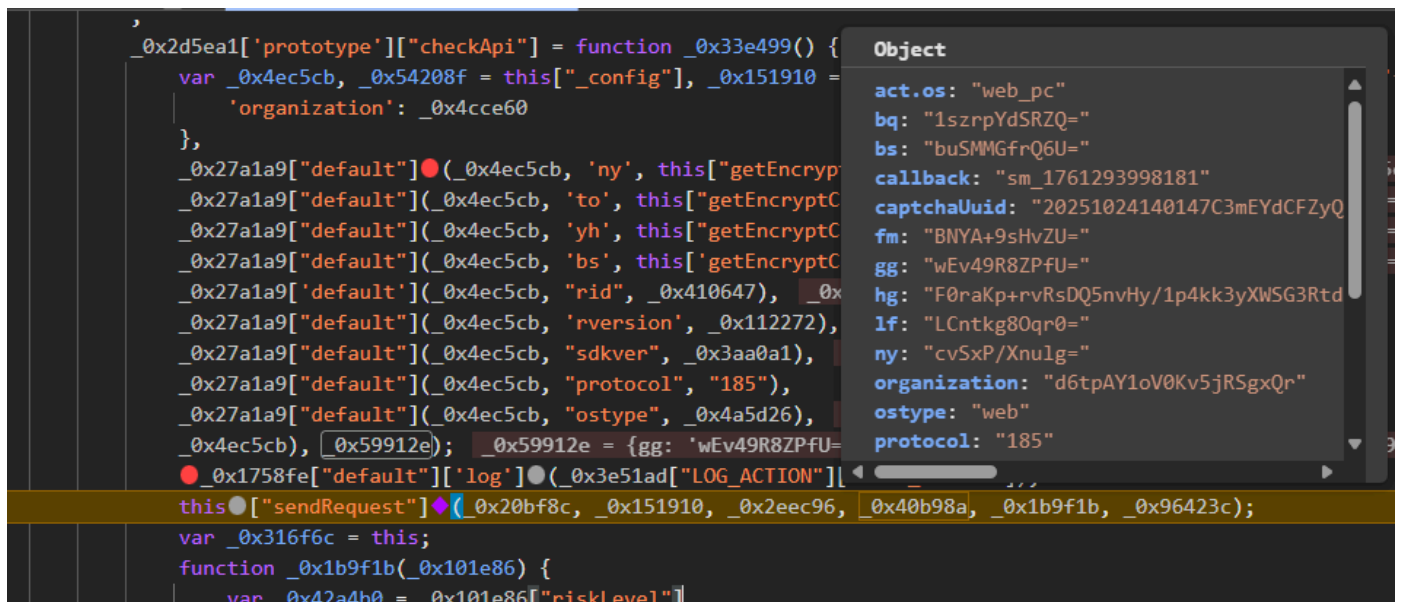
5.跟栈，刚刚我们跟到url的地方,可以找到_0x442533是在箭头处生成的。然后参数又在_0x1b14ab里，看函数和其他参数，这里应该是拼接url的，不会更改参数。



6.继续跟到这里,发现是传进来的，继续跟

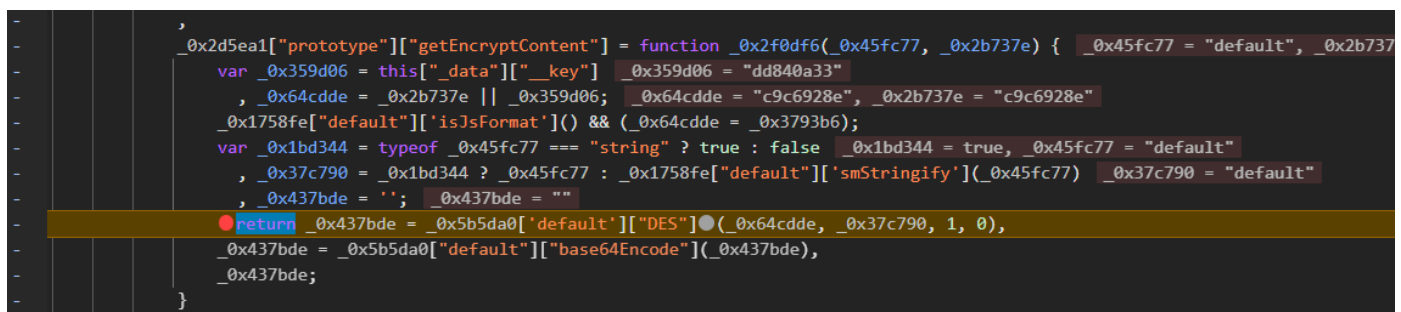
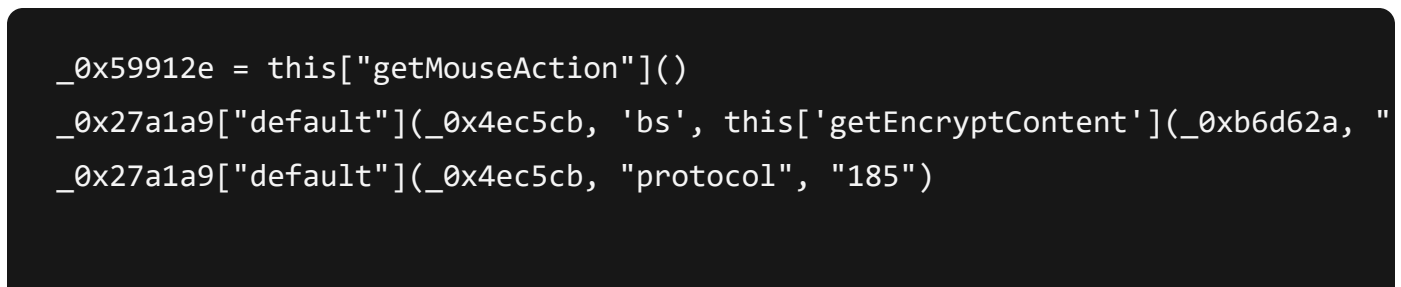
[Image blocked: unsafe source]

7.最后跟到这里，参数没有传进来且在函数内部找到了参数生成的地方。



8.发现我们要找的参数在_0x59912e, 以及一些赋值和加密, 找到rid是跟着图片一起传进来的。跟进去getEncryptContent看一看,DES三个写在哪里, 拿去试试.

就会发现"default"浏览器加密出来是 'cvSxP/Xnulg='和我們找到的在线加密的结果一致,测其他几个也一样的逻辑, 类似只是key/value变了。ny\to\yh等参数都是明的或des加密, 接下来看_0x59912e。



运算模式: ECB (电子密码本) 填充模式: Zeros 密钥长度: 64 bits

密钥: Text c9c6928e

default

字符编码: UTF-8 格式: Base64 (格式加密表示输出, 解密表示输入)

加密 解密 交换

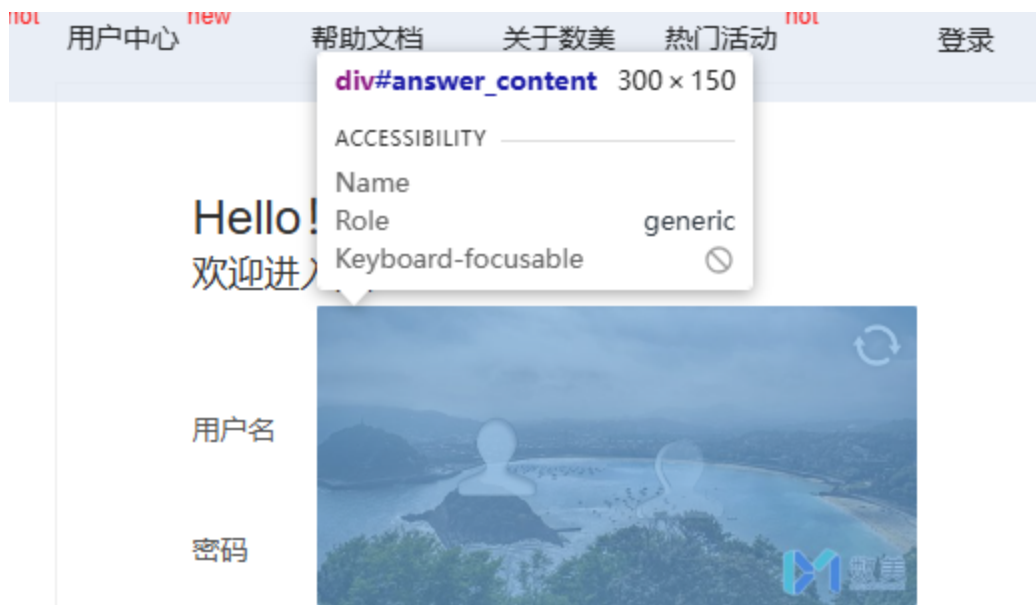
cvSxP/Xnulg=

9.跟进this["getMouseAction"], 可以看到有个switch, 根据第一行的 _0x484e75选择的, 这个就是我们的模式(滑块、点选之类的), 跟到滑块分支, 挨个打印参数, 可以猜测, gg是滑块的距离, hg是鼠标轨迹, th是滑块开始结束时间差, qt、lf未知.但qt=300 lf=150. 因为滑块肯定要跟图片关联, 我们看一下图片的大小. 恰好是300x150, 也就是qt、lf是图片的实际宽度.到这里, 主要参数的跟栈就跟完了, 接下来验证我们的推测.

```

-         break;
-         case "slide":
-             _0x58b708['gg'] = this["getEncryptContent"](_0x12c64a / _0x13da9a, "5129c2c2"),
-             _0x58b708['hg'] = this["getEncryptContent"](_0x13468f, "be221ccf"),
-             _0x58b708['th'] = this["getEncryptContent"](_0x5eae4a - _0x4dc2d8, '231a540d'),
-             _0x58b708['qt'] = this["getEncryptContent"](_0x13da9a, '38def2c1'),
-             _0x58b708['lf'] = this["getEncryptContent"](_0x46ac82, "7a8c235d"),
-             _0x58b708['act.os'] = _0x2abc3d;
-             _0x13da9a == 0 && (_0x58b708['gg'] = this["getEncryptContent"](0, "5129c2c2"));
-         break;

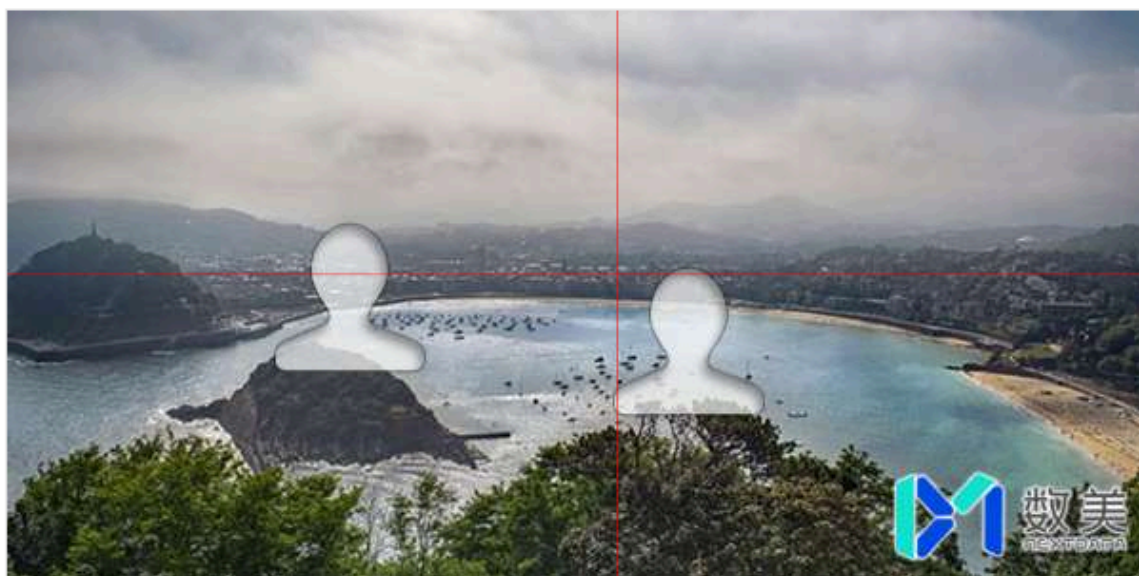
```



10.这个是我们刚刚看的滑块的参数, 300是宽度, 看轨迹最后也是160, 推测160是我们滑的位移. 接下来复制图片.因为我们复制出来是600宽度300高度的, 所以乘2.可以推测320应该是我们滑动图像左侧的位移距离.

```
> `${_0x12c64a}/${_0x13da9a}`
< '160/300'
> _0x13468f
< (6) [Array(3), Array(3), Array(3), Array(3), Array(3), Array(3)]
  ▶ 0: (3) [0, -16, 4]
  ▶ 1: (3) [148, -5, 364]
  ▶ 2: (3) [159, -8, 419]
  ▶ 3: (3) [160, -8, 518]
  ▶ 4: (3) [160, -8, 612]
  ▶ 5: (3) [160, -8, 721]
    length: 6
    [[Prototype]]: Array(0)
> _0x5eae4a - _0x4dc2d8
< 736
```

当前坐标: 320,138



11.往上翻一下, mouseEndX,mouseData都是在this["_data"]里面传过来的。

搜索mouseEndX, this["_data"]["mouseEndX"] = _0x11ba8c,也只有这一个给this._data赋值的, 打上断点.是把this._data.mouseMoveX赋值给了mouseEndX,搜mouseMoveX。可以看到this["_data"]["mouseMoveX"] 就是获取鼠标的位移, 然后一直覆盖。位移到这里也算找完了。

```
, _0x13468f = _0x488717["mouseData"] _0x13468f = (6) [Array(
, _0x4dc2d8 = _0x488717['startTime'] _0x4dc2d8 = 17612949173
, _0x5eae4a = _0x488717["endTime"] _0x5eae4a = 1761294918061
, _0x12c64a = _0x488717["mouseEndX"] _0x12c64a = 160
, _0x13da9a = _0x488717["trueWidth"] _0x13da9a = 300
, _0x46ac82 = _0x488717["trueHeight"] _0x46ac82 = 150
, _0x151169 = _0x488717["selectData"] _0x151169 = []
, _0x4f3a45 = _0x488717["blockWidth"] _0x4f3a45 = 40
```

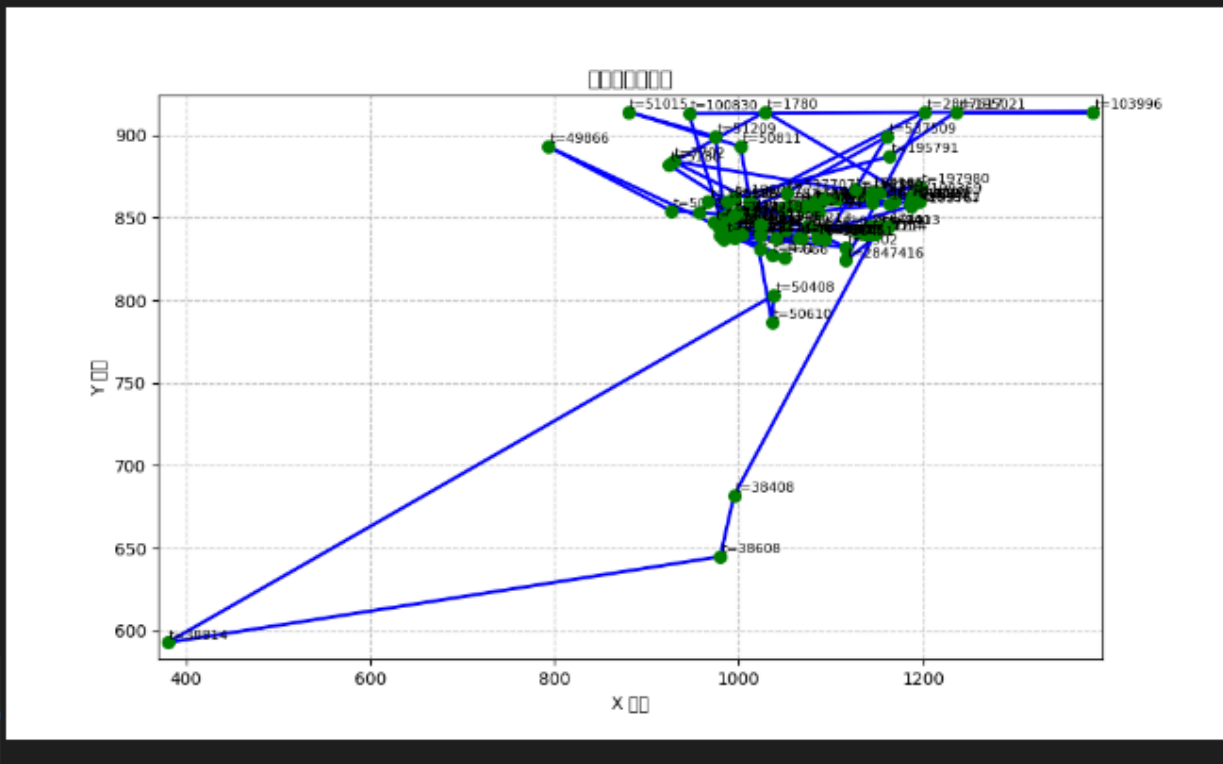
```
    _0x2d5ea1["prototype"]["endHandler"] = function _0x50f1c6() {
        this["_isMoving"] = false;
        var _0x5d91fe = this["_config"]["mode"] _0x5d91fe = "slide"
        , _0x37693b = this['_data'] _0x37693b = {selectData: Array(0), selectPosData: Array(0), errMsg: {...}, uuid: 'a36494
        , _0x5ef096 = _0x37693b["mousemoveX"] _0x5ef096 = 151
        , _0x3acca1 = _0x5ef096 === undefined ? 0 : _0x5ef096 _0x3acca1 = 151
        , _0x9f31d4 = _0x37693b["mousemoveY"] _0x9f31d4 = -8, _0x37693b = {selectData: Array(0), selectPosData: Array(0),
        , _0x21ac8c = _0x37693b["intervalTimer"] _0x21ac8c = 27179
        , _0x39782f = this["_data"] _0x39782f = {selectData: Array(0), selectPosData: Array(0), errMsg: {...}, uuid: 'a36494
        , _0x1e1915 = _0x39782f["blockWidth"] _0x1e1915 = 40
        , _0x17ffb8 = _0x1e1915 === undefined ? 0 : _0x1e1915 _0x17ffb8 = 40
        , _0x23ee25 = _0x39782f['slideWidth'] _0x23ee25 = 300, _0x39782f = {selectData: Array(0), selectPosData: Array(0),
        , _0x3d3fb3 = _0x23ee25 === undefined ? 0 : _0x23ee25 _0x3d3fb3 = 300
        , _0x11ba8c = _0x3acca1 _0x11ba8c = 151, _0x3acca1 = 151
    }
    mouseMoveX
```

```
    });
    else
        _0x5ccd0d <= 0 ? (this['setDomStyle'])(_0x50058b, { _0x5ccd0d = 190, _0x50058b = div#a3649412-24e8-4074-8210-8a
            'left': "0px"
        },
        _0x5ccd0d = 0) : (this["setDomStyle"])(_0x50058b, { _0x5ccd0d = 190, _0x50058b = div#a3649412-24e8-4074-8210-8a
            'left': _0x2fac96 + 'px' _0x2fac96 = 260
        },
        this["setDomStyle"](_0x34ab62, { _0x34ab62 = div#a3649412-24e8-4074-8210-8a1a442ba3fb-shumei_captcha_slide_pro
            'width': _0x2fac96 + _0xf83663 / 2 + 'px' _0x2fac96 = 260, _0xf83663 = 40
        },
        _0x5ccd0d = _0x2fac96); _0x5ccd0d = 190, _0x2fac96 = 260
    this["_data"]["mousemoveX"] = _0x5ccd0d,
    this['_data']['mousemoveY'] = _0x2dba4c;
```

12.接下来找鼠标轨迹.鼠标轨迹通常是监听获取并push到列表里面的,通常对document进行监听,搜mousemove, 17处找到下图这里。搜pageX或clientX,跟一下也可以找到下图这里。

看第一处,获取到鼠标轨迹列表.用python把他画下来,推测这个是我们鼠标在页面上的轨迹。

```
    this["_data"]["mousemoveData"] = _0x1076ce, _0x1076ce = Array(101)
    this["_data"]["mouseLeftClickData"] = _0x29d6f9, _0x29d6f9 = Array(32)
    this['_data']['mouseRightClickData'] = _0x35040f, _0x35040f = []
    this["_data"]["keyboardData"] = _0x35618f; _0x35618f = []
    switch (_0x56a60f) { _0x56a60f = "mousemove"
    case "mousemove":
        var _0x210d51 = this["_data"] _0x210d51 = {selectData: Array(0), selectPosData: Array(0), errMsg: {...}, uuid: 'a3649412-24
        , _0x427bfb = _0x210d51["fpMouseMoveX"] _0x427bfb = 991
        , _0x3f0eb2 = _0x210d51["fpMouseMoveY"]; _0x3f0eb2 = 842
        try {
            _0x1a7a28 = _0x1076ce["length"] - 1, _0x1a7a28 = 100, _0x1076ce = Array(101)
            _0x19076f = _0x1076ce[_0x1076ce['length'] - 1] ? _0x1076ce[_0x1076ce["length"] - 1][0] : 0; _0x19076f = 1066
        } catch (_0x21994b) {}
        _0x1076ce["length"] <= 100 && _0x427bfb && _0x3f0eb2 && _0x19076f != _0x427bfb ? (_0x1076ce['push'])(_0x427bfb, _0x3f0eb2,
        this["_data"]["mousemoveData"] = _0x1076ce) : clearInterval(_0x56a60f + "DataTimer"); _0x56a60f = "mousemove"
        break;
    case 'mouseLeftClick':
```

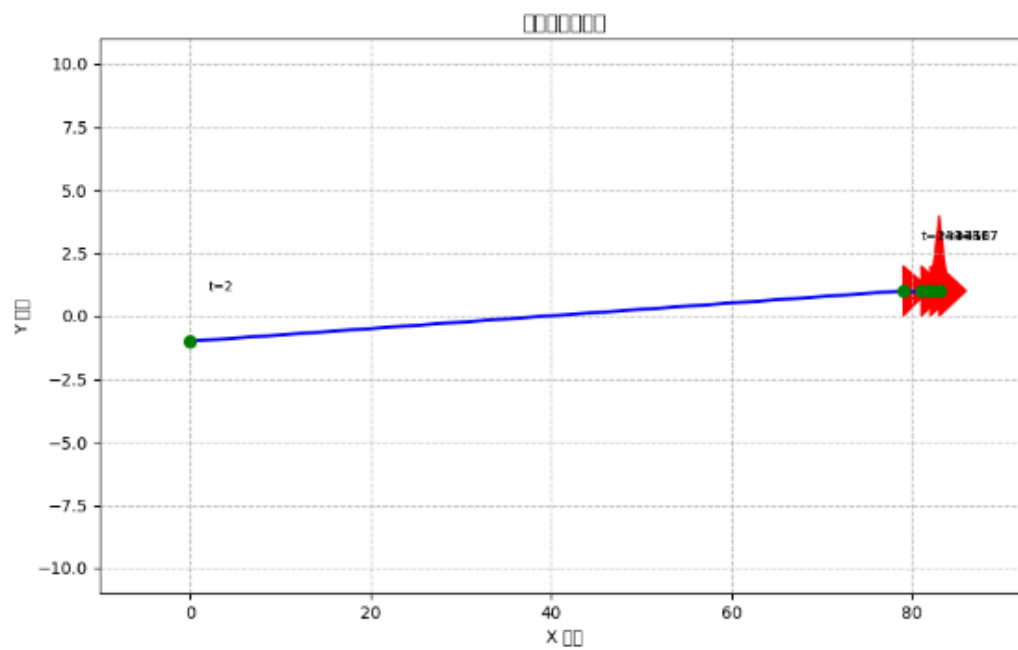
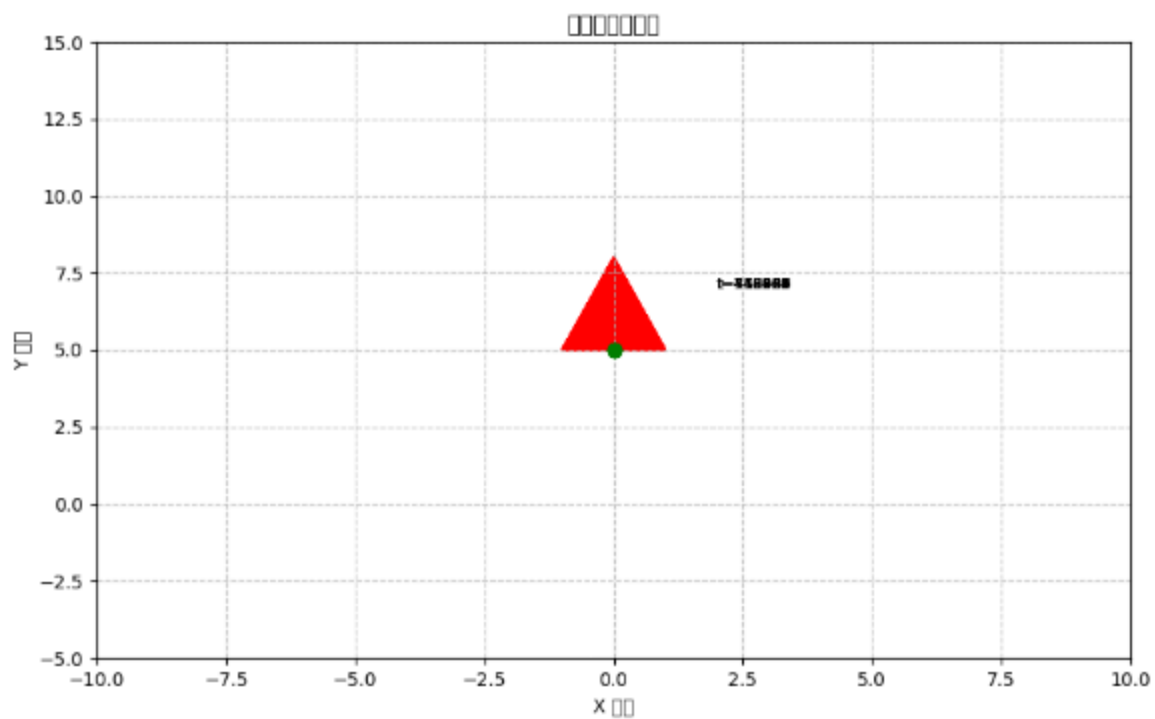


看第二处mousemoveX mouseY很明显了.获取鼠标轨迹并画下来.这个主要是我们没有滑动的原因，因为一直在被断，取消断点，打上记录点，看控制台打印,可以看到图三就正常了。就是我们滑动的轨迹.其次时间差也在上面。

```

    ,
    _0x2d5ea1["prototype"]["saveMouseData"] = function _0x3f2001() {
        var _0x132b17 = this['_data'] _0x132b17 = {selectData: Array(0), selectPosData: Array(0), errMsg: {...}, uuid: 'a3649412-
        , _0x44aafd = _0x132b17["startTime"] _0x44aafd = 1761296584201
        , _0x4e5f5f = _0x44aafd === undefined ? +new Date() : _0x44aafd _0x4e5f5f = 1761296584201
        , _0x538600 = _0x132b17['mousemoveX'] _0x538600 = 0, _0x132b17 = {selectData: Array(0), selectPosData: Array(0), errMsg:
        , _0x2f07b0 = _0x538600 === undefined ? 0 : _0x538600 _0x2f07b0 = 0
        , _0x56bdf9 = _0x132b17["mousemoveY"] _0x56bdf9 = 5, _0x132b17 = {selectData: Array(0), selectPosData: Array(0), errMsg:
        , _0x57a7e3 = _0x56bdf9 === undefined ? 0 : _0x56bdf9 _0x57a7e3 = 5
        , _0x5a0c1c = _0x132b17["mouseData"] _0x5a0c1c = Array(15), _0x132b17 = {selectData: Array(0), selectPosData: Array(0)
        , _0x23beac = _0x5a0c1c === undefined ? [] : _0x5a0c1c _0x23beac = Array(15)
        , _0x147b2d = _0x132b17["intervalTimer"] _0x147b2d = 30049, _0x132b17 = {selectData: Array(0), selectPosData: Array(0)
        , _0x34743e = +new Date(); _0x34743e = 1761297354649
        _0x23beac["length"] <= 100 ? (_0x23beac["push"])([_0x2f07b0, _0x57a7e3, _0x34743e - _0x4e5f5f]),
        this["data"]["mouseData"] = _0x23beac) : (clearInterval(_0x147b2d),
        this["data"]["intervalTimer"] = 0);
    }

```



13.到这里主要参数都找完了，还有一些其他的参数都是明文或DES加密明文生成的.我们需要生成轨迹、获取位移距离（需要识别验证码--暂未完成），之后组装发包,处理风控问题.